

INFORME DE ARQUITECTURA DE SOFTWARE

Architecture Report

Plataforma de Aula + Marketplace de Debate
Competitivo

VERSIÓN

1.0

FECHA

25 de junio de 2026

COMMIT

e1f7bf6

REPOSITORIO

github.com/YesIIBr0/OTR

GENERADO POR

Claude Code

CLASIFICACIÓN

Confidencial

Control del documento

Proyecto	OTR Academy — Plataforma de Aula + Marketplace de Debate Competitivo
Versión	1.0
Fecha	25 de junio de 2026
Commit	e1f7bf6cd1daa413b38f26e305b678a1e1be1e30
Repositorio	https://github.com/YesIIBr0/OTR
Estado	Final
Clasificación	Confidencial — uso interno del cliente
Audiencia	CTO · Ingeniería · Auditoría · Inversores

Historial de revisiones

Rev.	Fecha	Autor	Cambios
1.0	25 de junio de 2026	Claude Code	Emisión inicial — arquitectura al commit e1f7bf6

Tabla de Contenidos

1	Resumen Ejecutivo	5
1.1	Veredicto de arquitectura	5
1.2	Síntesis de la arquitectura	6
2	Arquitectura del Sistema	7
2.1	Estilo arquitectónico	7
2.2	Dominios (bounded contexts)	8
3	Arquitectura de Módulos	9
3.1	Módulos clave	9
3.2	El Aula como SPA por string-templates (ADR-0004)	10
4	Flujo de Datos y Render	11
4.1	Contrato de escape (seguridad de contenido)	11
5	Ciclo de Vida de la Petición	12
5.1	Convención de respuestas	12
6	Autenticación	13
6.1	Propiedades de la sesión	13
7	Autorización	15
7.1	Autorización por relación	15
8	Modelo de Datos	16
8.1	Doble schema Prisma (ADR-0002)	16
8.2	Migraciones	17
9	Superficie de API	18
10	Infraestructura y Despliegue	19

10.1	Imagen multi-stage	20
11	Integración y Entrega Continua	21
11.1	Gate de calidad	21
11.2	Despliegue iniciado por el VPS	21
12	Aspectos Transversales	23
12.1	Seguridad	23
12.2	Pagos y multimedia	23
13	Decisiones de Arquitectura	25
A	Glosario	26
B	Stack Tecnológico	27
C	Inventario de Dependencias	28
D	Variables de Entorno	29
E	Inventario de API	30
F	Hallazgos de Seguridad	31
G	Hallazgos de Rendimiento	32
H	Deuda Técnica	33
I	Mejoras Futuras	34
J	Referencias	35

01 VISIÓN GENERAL

Resumen Ejecutivo

OTR Academy es una plataforma vertical de educación en debate competitivo: combina un **LMS** (cursos, lecciones, quizzes, certificados), un **sistema de rating de debate** (Glicko-2 con adjudicación por coaches) y un **marketplace** de coaching con pagos en custodia (escrow). Está construida como un **monolito Next.js 15** desplegado en un solo contenedor sobre un VPS, con PostgreSQL al lado.

1 MONOLITO / DESPLIEGUE	48 MODELOS DE DATOS	~55 ENDPOINTS API	7 DEPENDENCIAS RUNTIME	4 ADRS FORMALES	100% CI TYPECHECK GATE
--------------------------------------	----------------------------------	-----------------------------	-------------------------------------	------------------------------	-------------------------------------

Este informe documenta la arquitectura del sistema en su estado al commit `e1f7bf6`: estilo arquitectónico, módulos, modelo de datos, ciclo de vida de las peticiones, autenticación y autorización, infraestructura, entrega continua y las decisiones de diseño que la sustentan, junto con hallazgos de seguridad y rendimiento, deuda técnica y recomendaciones.

1.1 Veredicto de arquitectura

La arquitectura es **deliberadamente simple y coherente con su escala** (1 desarrollador, objetivo ~3.000 usuarios). El monolito Next.js elimina el overhead de orquestación, la superficie de dependencias es mínima (7 paquetes en producción) y el pipeline de entrega es determinista. Las decisiones están documentadas como ADRs con sus trade-offs explícitos. Los principales riesgos son conocidos y acotados: un único punto de fallo (SPOF), la trampa del *dobles schema Prisma*, y la falta de type-safety en la capa de UI (SPA por string-templates). Ninguno es bloqueante a la escala actual; todos tienen una vía de mitigación trazada en el roadmap.

✓ Apto para producción a su escala

El sistema es operable por una sola persona, los despliegues son reproducibles y existen barreras automáticas (typecheck + validación de schema) que impiden desplegar código roto. La ruta a 3.000+ usuarios (cluster, PgBouncer, Redis) no exige reescritura.

1.2 Síntesis de la arquitectura

Dimensión	Decisión	ADR
Topología	Monolito Next.js (App Router): landing + SPA + API en un contenedor	ADR-0001
Persistencia	PostgreSQL 16; doble schema Prisma (SQLite dev / Postgres prod)	ADR-0002
Entrega	CI construye imagen → ghcr.io; el VPS la <i>baja</i> por cron (sin SSH entrante)	ADR-0003
UI interna (Aula)	SPA vanilla por string-templates dentro de Next (no componentes React)	ADR-0004
Pagos	Stripe Checkout + webhooks; custodia (escrow) para bookings de coaching	—
Multimedia	Cloudflare Stream para vídeo; uploads en volumen fuera de la imagen	—

02 VISTA LÓGICA

Arquitectura del Sistema

OTR adopta un **monolito modular**: un único proceso Next.js que sirve tres superficies — el sitio público, la aplicación interna ("Aula") y la API — organizadas en capas claras dentro de la misma base de código.

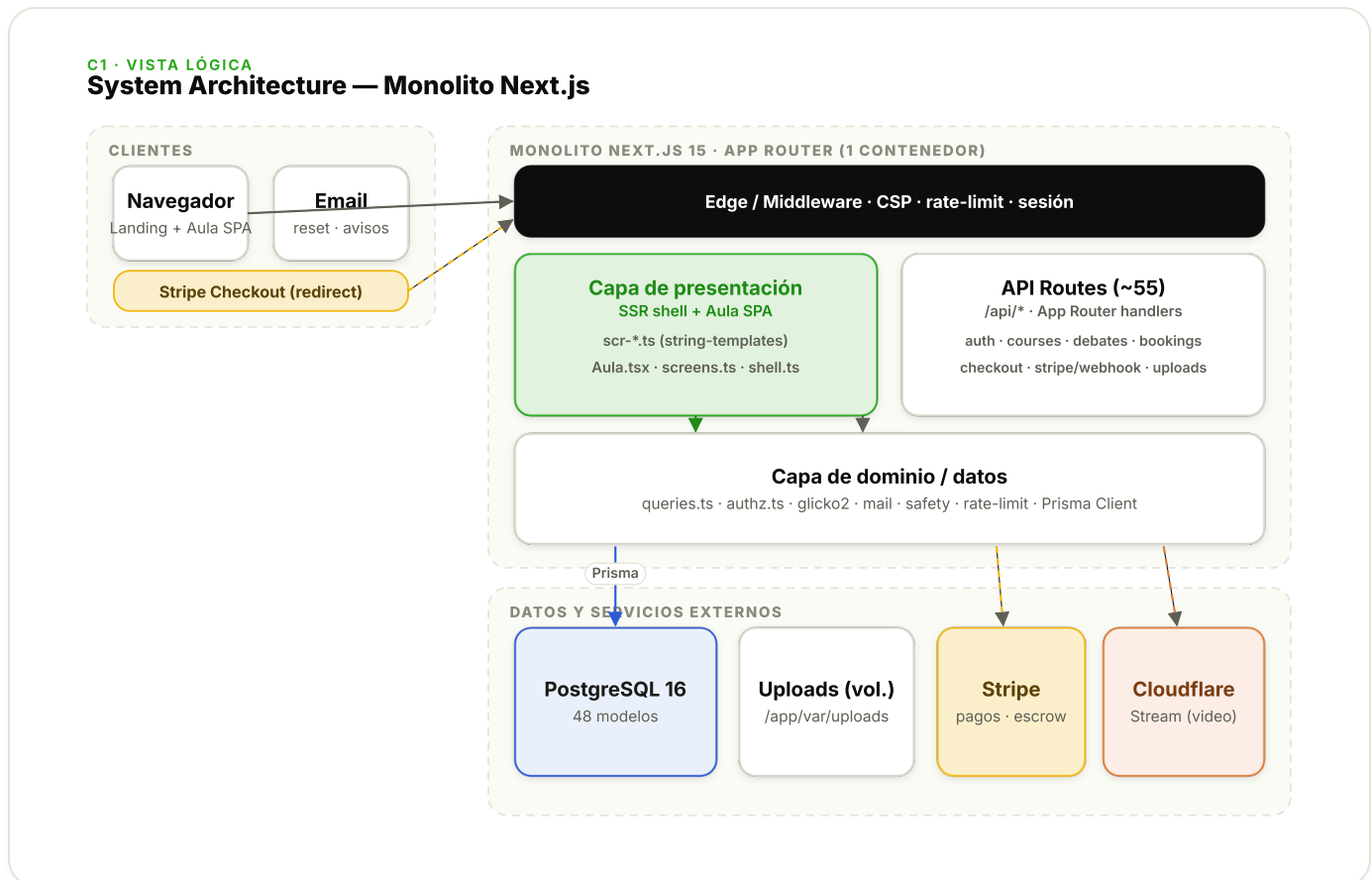


Figura 1. Arquitectura lógica del sistema: clientes, monolito en capas y servicios de datos/externos.

2.1 Estilo arquitectónico

El sistema sigue un patrón de **capas** sobre el App Router de Next.js: (1) borde/middleware (CSP, rate-limit, resolución de sesión); (2) presentación (shell SSR + la SPA del Aula); (3) API como *route handlers*; (4) dominio y datos (consultas, autorización, reglas de negocio, Prisma). La comunicación entre capas es por llamada de función dentro del mismo proceso — sin red interna, sin serialización — lo que mantiene la latencia baja y el modelo mental simple.

§

Por qué un monolito

A escala de 1 desarrollador y ~3.000 usuarios, los límites de servicio de los microservicios añadirían coste operativo sin beneficio. La Ley de Conway aplica: 1 dev = 1 servicio. El monolito es suficiente con cluster + Redis + PgBouncer cuando llegue el momento (ver Roadmap), sin reescritura.

2.2 Dominios (bounded contexts)

Aunque físicamente es un monolito, el código está organizado por dominios de negocio cohesivos:

Dominio	Responsabilidad	Entidades núcleo
Identidad	Usuarios, roles, niveles, skills, tutela parental	User, Level, StudentSkill, Guardianship
Aprendizaje	Cursos, módulos, lecciones, quizzes, entregas, certificados	Course, Module, Lesson, Quiz, Enrollment
Debate & Rating	Rondas, ballots, rúbricas, rating Glicko-2, torneos	DebateRecord, Ballot, RatingUpdate, Tournament
Marketplace	Perfiles de coach, paquetes, reservas, escrow, sesiones	CoachProfile, Booking, EscrowTxn
Comunidad	Foros, conversaciones, mensajería, notificaciones	ForumThread, Conversation, Notification
Moderación	Reportes, suspensión, seguridad de menores	Report, safety.ts

03

COMPONENTES INTERNOS

Arquitectura de Módulos

La lógica de negocio vive en `app/Lib/` : un conjunto de módulos con responsabilidad única que los *route handlers* y la UI consumen.

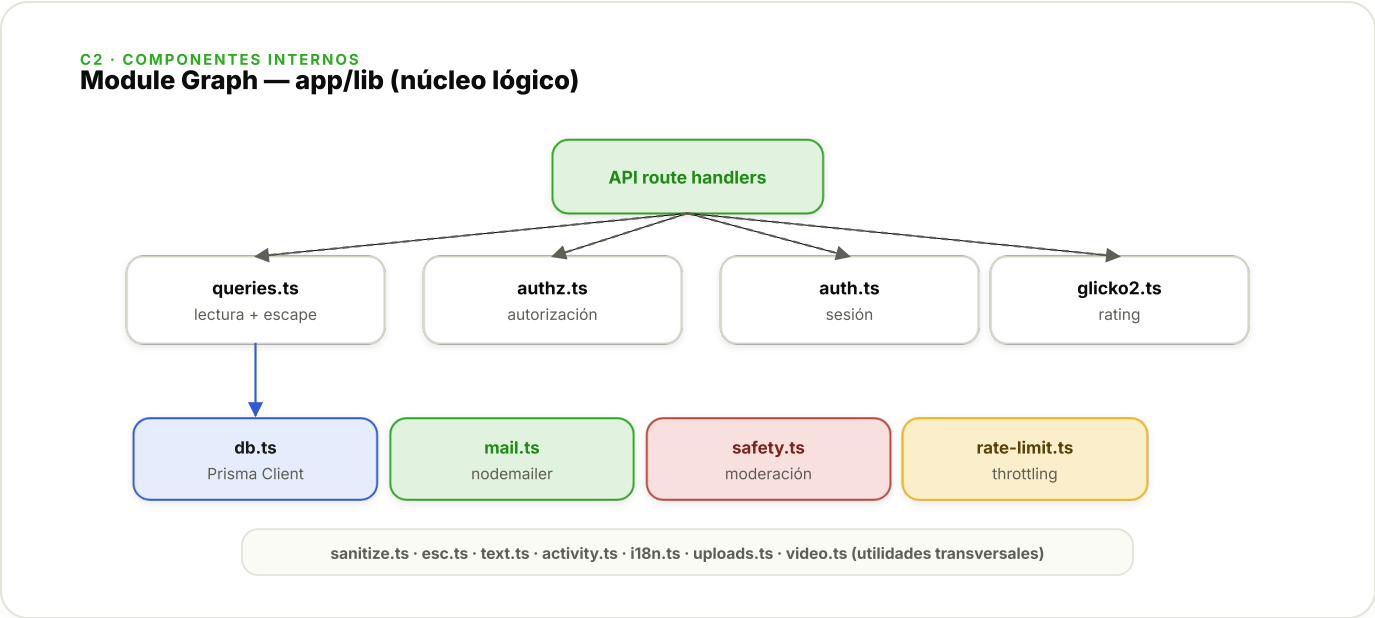


Figura 2. Grafo de módulos de `app/lib`: handlers de API sobre la capa de consultas/autorización, apoyados en utilidades transversales.

3.1

Módulos clave

Módulo	Rol
<code>db.ts</code>	Cliente Prisma singleton (evita múltiples pools en dev)
<code>queries.ts</code>	Lecturas + contrato de escape : escapa el texto de usuario una sola vez
<code>auth.ts</code> / <code>auth-crypto.ts</code>	Sesión por cookie firmada; scrypt para contraseñas
<code>authz.ts</code>	Autorización por relación (coach↔alumno, padre↔hijo)
<code>glicko2.ts</code>	Motor de rating Glicko-2 (rating, RD, volatilidad, tier)
<code>safety.ts</code> / <code>sanitize.ts</code>	Moderación y saneamiento HTML (anti-XSS)
<code>rate-limit.ts</code>	Throttling de endpoints sensibles
<code>mail.ts</code>	Envío transaccional vía nodemailer/SMTP

3.2 El Aula como SPA por string-templates (ADR-0004)

La aplicación interna se portó de un prototipo HTML/JS. En lugar de reescribirla en componentes React, cada pantalla es una función que devuelve un *string* de HTML (`app/lib/scr-*.ts`); un router propio (`screens.ts`) y un único componente React (`Aula.tsx`) hacen `root.innerHTML = ...` y delegan los clicks por atributos `data-*`. El estado vive en `window.DB`.

Cada pantalla es una función pura $f(\text{estado}) \rightarrow \text{HTML}$. Los eventos se delegan por `data-*`.

```
// app/lib/scr-debate.ts (patrón string-template)
export function viewDebateHub(DB: AppData): string {
  return `<section class="hub">
    <h1 class="page-title">${t("debate.title")}</h1>
    ${DB.debates.map(recordRow).join("")}
    <button class="btn btn-primary" data-action="debate-record">
      ${t("debate.recordDebate")}
    </button>
  </section>`;
}
```



Deuda consciente

Los `scr-*.ts` llevan `// @ts-nocheck` (sin type-safety en la UI) y re-renderizan por `innerHTML` completo. Sin code-splitting, `/aula` son ~206 kB de First Load. Aceptable a esta escala; la mitigación (migración incremental a componentes + split por ruta) está trazada pero no es urgente.

04 CAPA DE PRESENTACIÓN Flujo de Datos y Render

El render del Aula es un ciclo determinista: navegación → carga de datos → función de pantalla → inyección de HTML → delegación de eventos.

C3 · ADR-0004

Data Flow — render del Aula (SPA string-template)

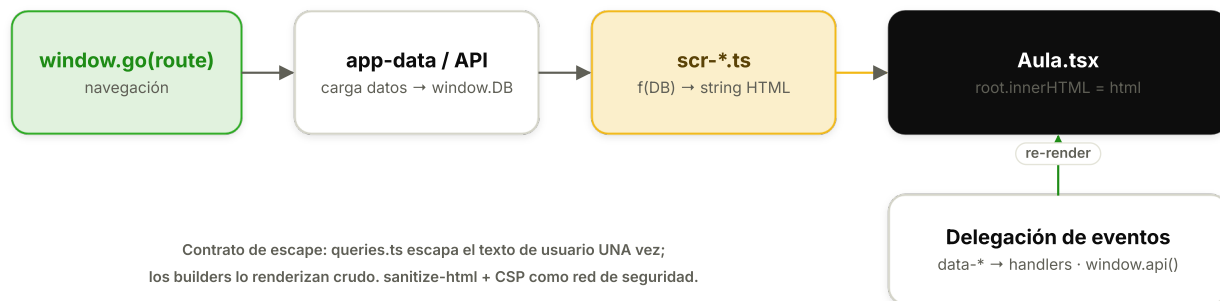


Figura 3. Flujo de render de la SPA: window.go() carga datos en window.DB, scr-*.ts produce HTML y Aula.tsx lo inyecta.

4.1 Contrato de escape (seguridad de contenido)

La regla invariante del sistema: `queries.ts` **escapa el texto de usuario exactamente una vez**; los *builders* `scr-*.ts` lo renderizan crudo. Romper el contrato por ambos lados causa o bien doble-escape (`&` visible) o bien XSS. La red de seguridad es doble: `sanitize-html` sobre contenido rico y una **Content-Security-Policy** en el borde.

§ i18n

Los textos viven en `app/lib/i18n-keys/*.ts` como diccionarios `{ es, en }` por dominio, resueltos por el helper `t()`. La UI no contiene strings hardcodeados de cara al usuario.

05 CAMINO DE DATOS

Ciclo de Vida de la Petición

Toda petición a la API recorre el mismo camino con cortes de control consistentes.

C4 · CAMINO DE UNA PETICIÓN

Request Flow — del navegador a Postgres



Respuesta JSON ← (camino inverso) · errores normalizados por lib/api.ts ok()/bad()
 getSessionUser() resuelve la cookie firmada en cada handler antes de tocar datos

Figura 4. Camino de una petición: del navegador a Postgres y vuelta, con resolución de sesión por handler.

5.1 Convención de respuestas

Los *route handlers* normalizan las respuestas mediante `lib/api.ts: ok(data)` devuelve `{ ok: true, ...data }` y `bad(error, status)` devuelve `{ ok: false, error }`. La entrada se sanea con `clean()` y `readJson()` (que nunca lanza ante un body malformado).

Estructura canónica de un handler: sesión → validación → dominio → respuesta normalizada.

```

export async function POST(req: Request) {
  const user = await getSessionUser(); // 1) sesión
  if (!user) return bad("No autenticado", 401); // 2) authz base
  const body = await readJson<Payload>(req); // 3) entrada segura
  const result = clean(body.result, 8).toUpperCase();
  if (!VALID_RESULTS.has(result)) return bad("result inválido");
  // 4) regla de dominio + Prisma ... 5) ok({...})
}
  
```

06

IDENTIDAD

Autenticación

La sesión es una **cookie firmada** (`otr_session`), resuelta en el servidor en cada petición. No hay JWT de cliente ni almacenamiento de sesión en BD.

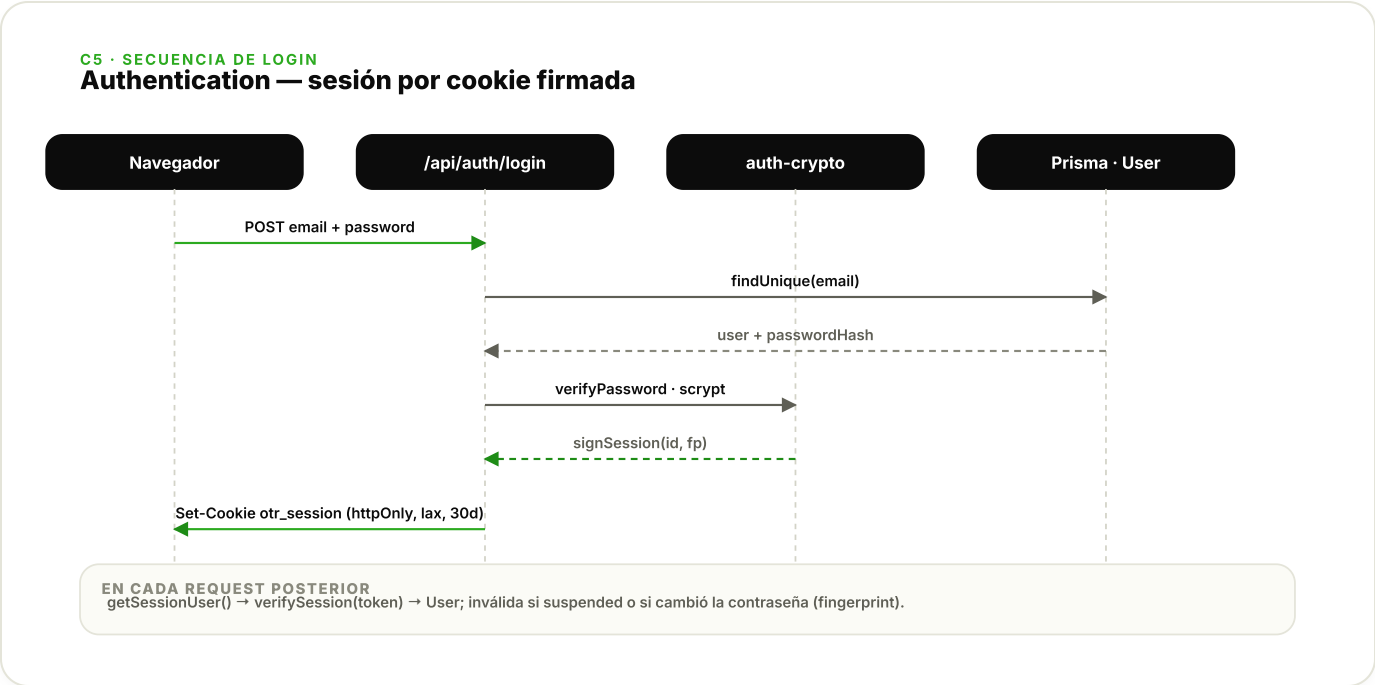


Figura 5. Secuencia de login y emisión de la cookie de sesión firmada.

6.1

Propiedades de la sesión

Propiedad	Valor	Motivo
Cookie	<code>otr_session</code> · httpOnly · sameSite=lax	No accesible a JS; mitiga CSRF básico
Secure	Sólo en producción (HTTPS)	Evita fuga en tránsito
Firma	<code>signSession(id, fp)</code> con <code>AUTH_SECRET</code>	Integridad; no es manipulable por el cliente
Vínculo a contraseña	<i>fingerprint</i> del hash en el token	Cambiar la contraseña invalida sesiones viejas
Vida	30 días	Balance UX/seguridad
Revocación	<code>user.suspended</code> → sin sesión válida	Expulsión inmediata por moderación

Resolución de sesión en cada request — falla cerrado ante suspensión o cambio de contraseña.

```
export async function getSessionUser() {
  const token = (await cookies()).get("otr_session")?.value;
  if (!token) return null;
  const session = verifySession(token);
  if (!session) return null;
  const user = await db.user.findUnique({ where: { id: session.userId } });
  if (!user || user.suspended) return null; // expulsión por moderación
  if (passwordFingerprint(user.passwordHash) !== session.fp) return null; // pwd cambió
  return user;
}
```

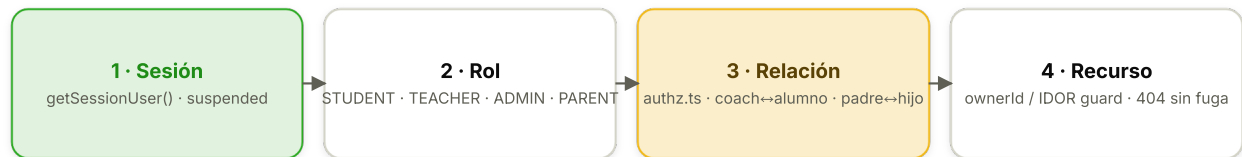
07 CONTROL DE ACCESO

Autorización

La autorización es **defensa en profundidad**: cuatro capas, de la sesión al recurso concreto.

C6 · DEFENSA EN PROFUNDIDAD

Authorization — capas de control de acceso



Ej.: adjudicar un debate exige rol TEACHER/ADMIN + vínculo (reserva o inscripción) con el alumno;
el menor nunca recibe escritura de rating sin coach vinculado (T&S §7.4).

Figura 6. Capas de autorización: sesión → rol → relación → recurso.

7.1 Autorización por relación

Más allá del rol, las acciones sensibles exigen un **vínculo** verificado. Ejemplo real (adjudicación de debate): un **TEACHER** sólo puede mover el rating de un alumno con el que tiene una *reserva* o que está inscrito en su curso; un **ADMIN** puede a cualquiera. Esto materializa la política de seguridad de menores (T&S §7.4): nadie escribe en el rating de un menor sin vínculo de coaching.



Guardas IDOR

Los recursos por id devuelven 404 (no 403) cuando el solicitante no es el dueño, evitando filtrar la existencia de recursos ajenos.

08 PERSISTENCIA

Modelo de Datos

El dominio se modela en **48 entidades Prisma**. El subconjunto núcleo conecta identidad, aprendizaje, debate y marketplace.

C9 · MODELO DE DATOS

Database ERD — entidades núcleo (subconjunto de 48)

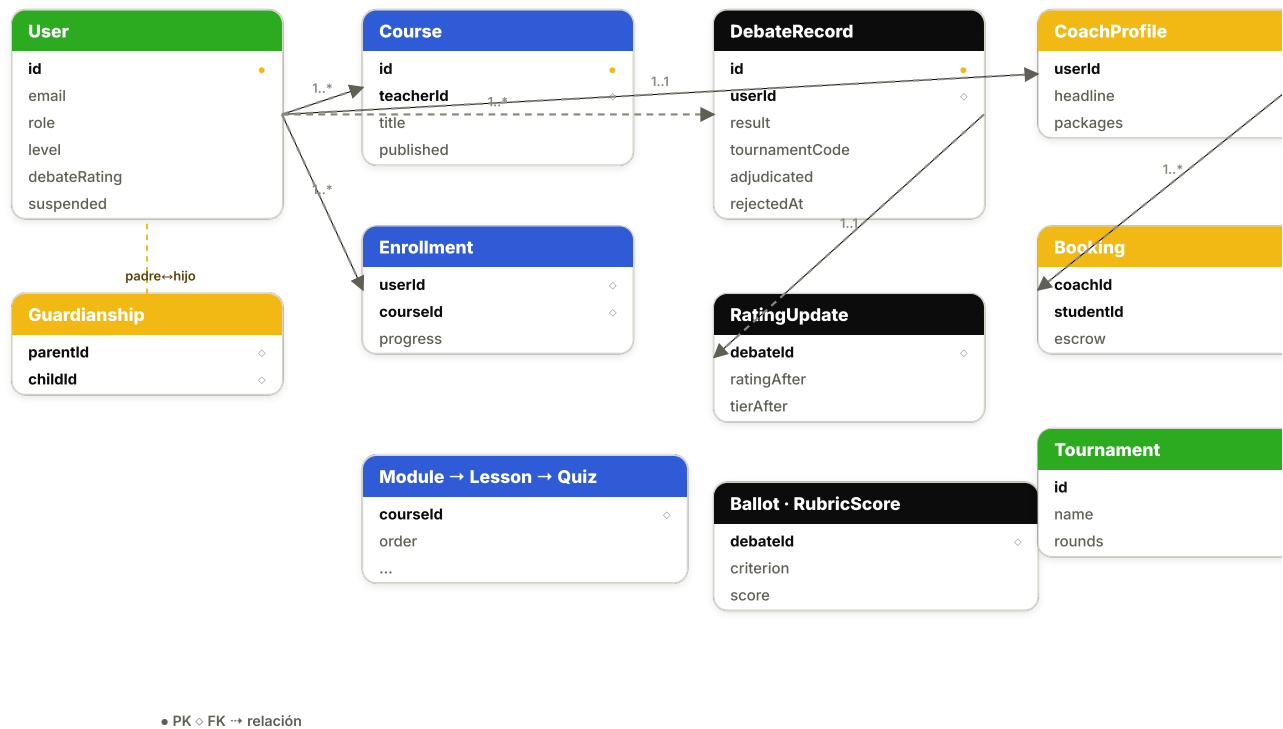


Figura 7. ERD del subconjunto núcleo (de 48 modelos): identidad, aprendizaje, debate/rating y marketplace.

8.1 Doble schema Prisma (ADR-0002)

Se mantienen dos schemas idénticos: `schema.prisma` (provider `sqlite`, para dev instantáneo) y `schema.postgres.prisma` (provider `postgresql`, producción). El build de CI hace `cp schema.postgres.prisma schema.prisma` antes de `prisma generate`.



Trampa del doble schema

Ambos schemas deben editarse a mano de forma idéntica; olvidarlo causa *drift*. La alternativa recomendada (single source of truth: sólo Postgres, con Docker Compose local en dev) elimina la clase entera de bugs y está pendiente de priorización.

8.2 Migraciones

La producción aplica cambios de esquema con `prisma db push` (sin carpeta de migraciones): el redeploy ejecuta `db push --skip-generate`, que añade columnas/índices aditivos de forma idempotente. Los cambios de **datos** (no de esquema) requieren SQL manual.

09 COMUNICACIÓN

Superficie de API

La API expone ~55 *route handlers* del App Router, agrupados por dominio. Todos comparten la convención de respuesta y la resolución de sesión por handler.

C10 · COMUNICACIÓN

API Surface — ~55 route handlers por dominio

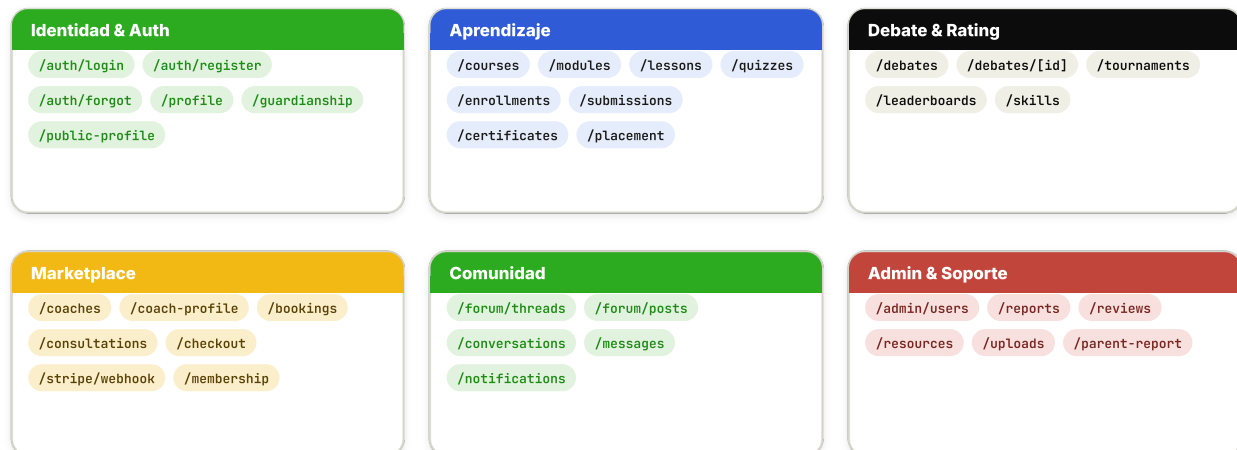


Figura 8. Mapa de la superficie de API por dominio (subconjunto representativo de ~55 endpoints).

El inventario completo de endpoints se lista en el **Apéndice E**. Los webhooks de Stripe (`/api/stripe/webhook`) verifican la firma con `STRIPE_WEBHOOK_SECRET` antes de procesar eventos de pago.

10

PLANO FÍSICO

Infraestructura y Despliegue

La app corre como un contenedor Docker tras Nginx (TLS) sobre un VPS de 1 vCPU, junto a un contenedor PostgreSQL, orquestados por Docker Compose.

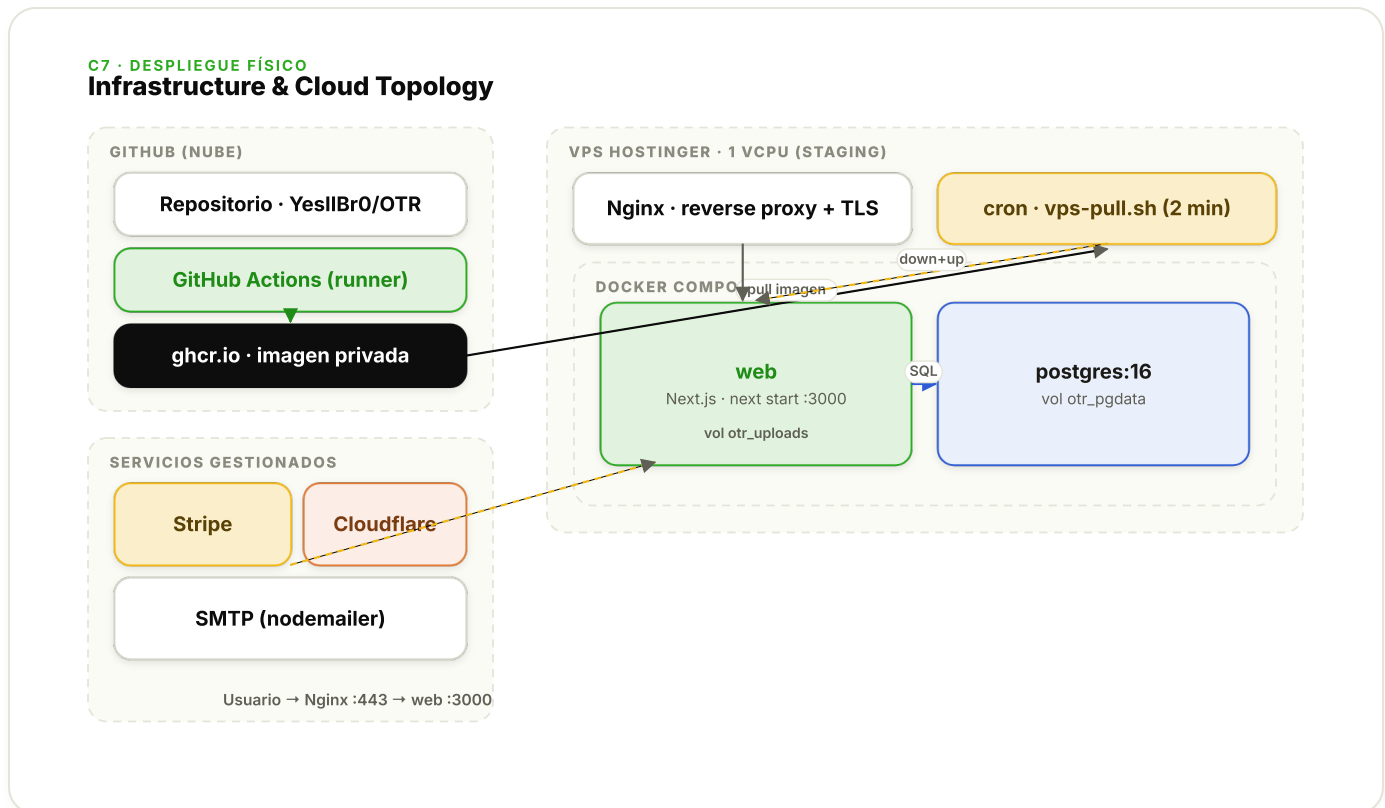


Figura 9. Topología de infraestructura: GitHub/ghcr, VPS con Nginx + Docker Compose, y servicios gestionados.

10.1 Imagen multi-stage

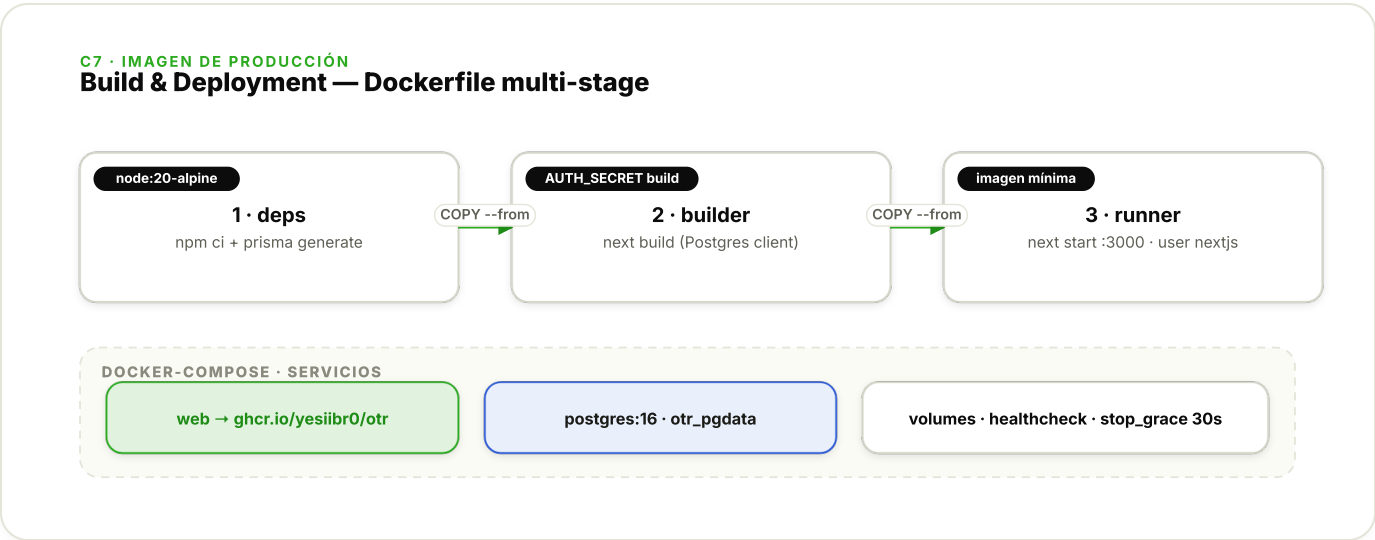


Figura 10. Build de la imagen en tres etapas (deps → builder → runner) y servicios de Compose.

El `Dockerfile` usa tres etapas sobre `node:20-alpine`: **deps** (instala con `npm ci + prisma generate`), **builder** (`next build` con el cliente Prisma de Postgres) y **runner** (imagen mínima que arranca `next start` como usuario sin privilegios `nextjs`). Los uploads viven en un volumen (`otr_uploads`) fuera de la imagen y de `public/`.

Servicio	Imagen	Notas
web	ghcr.io/yesibr0/otr:latest	Expuesto en 127.0.0.1:3000; <code>stop_grace_period: 30s</code> (drena requests)
postgres	postgres:16-alpine	Volumen <code>otr_pgdata</code> ; puerto no expuesto al host
nginx (host)	—	Reverse proxy + TLS al puerto 443

✓

Healthcheck que recupera

El contenedor `web` tiene un healthcheck HTTP contra `/aula` : si la app se cuelga (no sólo si el proceso muere), Docker la reinicia.

11

PIPELINE

Integración y Entrega Continua

El pipeline (ADR-0003) invierte el flujo habitual: GitHub construye y publica la imagen; el **VPS la baja por cron**. Nunca hay SSH entrante.

CS · ENTREGA CONTINUA

CI/CD Pipeline — "VPS-pull" (ADR-0003)

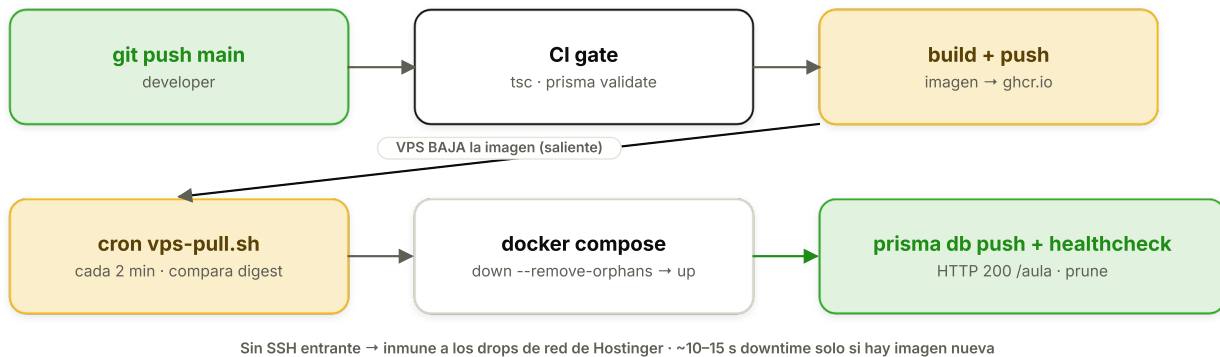


Figura 11. Pipeline de CI/CD "VPS-pull": gate de CI → build → ghcr.io → cron en el VPS → redeploy.

11.1 Gate de calidad

Cada push y PR ejecuta el job `ci`: instala, genera el cliente Prisma, **valida ambos schemas** y corre `tsc --noEmit`. Sólo si pasa, el job `build-and-push` (en push a `main`) construye la imagen y la publica en ghcr.io con tags `:latest` y `:<sha>`.

El gate de CI impide desplegar código que no compila o con un schema inválido.

```

ci:
  steps:
    - run: npm ci
    - run: npx prisma generate
    - name: Validar ambos schemas Prisma
      run: |
        npx prisma validate
        npx prisma validate --schema prisma/schema.postgres.prisma
    - name: Typecheck (tsc --noEmit)
      run: npx tsc --noEmit
  
```

11.2 Despliegue iniciado por el VPS

Un cron corre `scripts/vps-pull.sh` cada ~2 min: compara el digest de la imagen y, si cambió, hace `docker compose down --remove-orphans && up -d`, aplica `prisma db push` y verifica un `HTTP 200` en

`/aula` antes de podar imágenes viejas. Como el VPS **inicia** la conexión saliente, el flujo es inmune a los drops de red de Hostinger sobre IPs entrantes de runners de GitHub.

§**Coste del determinismo**

`down+up` implica ~10–15 s de downtime sólo cuando hay imagen nueva — se eligió sobre el *rolling recreate* tras dos incidentes de “container name already in use”. Mitigado con `stop_grace_period` para drenar peticiones en curso.

12

CALIDAD DE SERVICIO

Aspectos Transversales

Preocupaciones que cruzan todas las capas: seguridad, pagos, multimedia, correo y observabilidad.

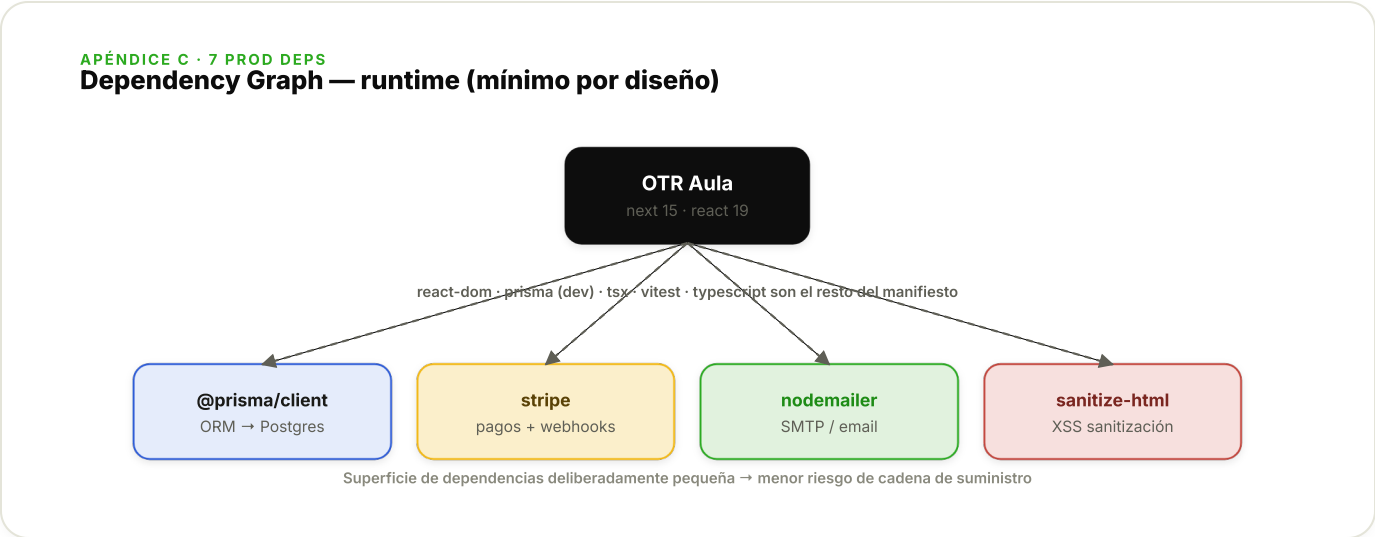


Figura 12. Grafo de dependencias de runtime: superficie mínima por diseño (7 paquetes de producción).

12.1

Seguridad

Control	Mecanismo
XSS	Contrato de escape + <code>sanitize-html</code> + CSP en el borde
Sesión	Cookie firmada httpOnly, vinculada a la contraseña; revocación por suspensión
Contraseñas	<code>scrypt</code> (auth-crypto); fingerprint del hash en la sesión
Abuso	<code>rate-limit.ts</code> en endpoints sensibles (login, reset, etc.)
Pagos	Verificación de firma de webhook Stripe; custodia (escrow) en bookings
Menores	Tutela parental (Guardianship) + authz por relación; seguridad de menores T&S
Contenedor	Usuario sin privilegios <code>nextjs</code> ; secretos por variables de entorno

12.2

Pagos y multimedia

Stripe gestiona los pagos vía Checkout (redirect) y webhooks firmados; el marketplace mantiene los fondos en **escrow** hasta la confirmación de la sesión de coaching. El **vídeo** se sirve por **Cloudflare**

Stream (no se almacenan binarios pesados en el VPS). El correo transaccional sale por SMTP vía `nodemailer`.

13

ADRS

Decisiones de Arquitectura

Las decisiones estructurales están registradas como ADRs versionados en `docs/adr/`, cada uno con contexto, decisión y consecuencias.

ADR	Decisión	Estado
0001	Monolito Next.js (no microservicios)	Aceptado
0002	Doble schema Prisma (SQLite dev / Postgres prod)	Aceptado (con deuda)
0003	CI/CD "VPS-pull" (sin SSH entrante)	Aceptado
0004	Aula como SPA vanilla por string-templates	Aceptado (con deuda)



Higiene de decisiones

Cada ADR declara explícitamente "Revisar si...", fijando el disparador que obligaría a reconsiderar la decisión (p. ej. crecer a 5+ devs, o que un subsistema necesite escalar de forma independiente).



APÉNDICE A

Glosario

Término	Definición
ADR	Architecture Decision Record — registro versionado de una decisión y sus consecuencias
Aula	La aplicación interna (SPA) que usan alumnos, coaches, padres y admins
Escrow	Custodia: el pago de una reserva se retiene hasta confirmar la sesión
Glicko-2	Sistema de rating (rating, RD, volatilidad) usado para el ranking de debate
Ballot	Hoja de adjudicación de un juez: resultado + rúbrica por criterio
VPS-pull	Patrón de despliegue donde el servidor baja la imagen (no GitHub la empuja)
SPOF	Single Point of Failure — punto único de fallo
IDOR	Insecure Direct Object Reference — acceso a recursos ajenos por id
SSR	Server-Side Rendering — render del shell en el servidor

B

APÉNDICE B

Stack Tecnológico

Capa	Tecnología	Versión
Framework	Next.js (App Router)	15.1.6
UI	React / React-DOM	19.0.0
Lenguaje	TypeScript	5.7.3
ORM	Prisma + @prisma/client	6.19.3
Base de datos	PostgreSQL (prod) / SQLite (dev)	16 / 3
Pagos	Stripe	22.2.0
Email	nodemailer	8.0.10
Sanitización	sanitize-html	2.17.4
Tests	Vitest	4.1.9
Runtime	Node.js	20 (alpine)
Contenedor	Docker + Docker Compose	multi-stage
Proxy / TLS	Nginx	host
Registro de imágenes	GitHub Container Registry (ghcr.io)	—
Vídeo	Cloudflare Stream	—

C

APÉNDICE C

Inventario de Dependencias

Producción (7): @prisma/client, next, react, react-dom, nodemailer, sanitize-html, stripe.

Desarrollo (9): prisma, typescript, tsx, vitest, @types/node, @types/react, @types/react-dom, @types/nodemailer, @types/sanitize-html.



Riesgo de cadena de suministro bajo

Sólo 7 dependencias de runtime, todas de mantenedores de primer nivel (Vercel, Prisma, Stripe).
Una superficie pequeña reduce la exposición a vulnerabilidades transitivas.

D

APÉNDICE D

Variables de Entorno

Variable	Propósito	Sensible
DATABASE_URL	Cadena de conexión a PostgreSQL	Sí
AUTH_SECRET	Firma de la cookie de sesión	Sí
APP_URL	URL pública de la app (enlaces de email, redirects)	No
SMTP_URL / MAIL_FROM	Transporte SMTP y remitente	Sí / No
STRIPE_SECRET_KEY	Clave secreta de Stripe	Sí
STRIPE_WEBHOOK_SECRET	Verificación de firma de webhooks	Sí
CLOUDFLARE_CUSTOMER_SUBDOMAIN	Entrega de vídeo (Stream)	No
CLOUDFLARE_ACCOUNT_ID / API_TOKEN	Gestión de Cloudflare Stream	Sí
UPLOAD_DIR	Ruta del volumen de uploads	No
POSTGRES_USER / PASSWORD / DB	Credenciales del contenedor Postgres	Sí
SEED_PASSWORD / SEED_FORCE / SEED_ALLOW_PROD	Control del seed de datos demo	Sí

E

APÉNDICE E

Inventario de API

/api/admin/users /api/app-data /api/auth/forgot /api/auth/login /api/auth/logout
/api/auth/register /api/auth/reset /api/bookings /api/bookings/[id] /api/certificates
/api/checkout /api/coach-profile /api/coaches /api/coaches/[id] /api/consultations
/api/consultations/[id] /api/consultations/availability /api/conversations /api/courses
/api/courses/[id] /api/debates /api/debates/[id] /api/enrollments /api/forum/posts
/api/forum/threads /api/guardianship /api/leaderboards /api/lesson-progress /api/lessons
/api/lessons/[id] /api/lessons/reorder /api/membership /api/messages /api/modules
/api/modules/[id] /api/modules/reorder /api/notifications /api/parent-report
/api/placement /api/profile /api/public-profile /api/quiz-attempts /api/quizzes
/api/quizzes/[id] /api/quizzes/[id]/attempt /api/reports /api/resources /api/reviews
/api/skills /api/stripe/webhook /api/submissions /api/submissions/[id] /api/tournaments
/api/uploads

Total: 54 grupos de *route handlers* (cada uno expone los verbos HTTP correspondientes: GET/POST/PATCH/DELETE).

F

APÉNDICE F

Hallazgos de Seguridad

#	Hallazgo	Severidad	Estado / Mitigación
S1	XSS si se omite <code>esc()</code> / <code>sanitizeHtml()</code> en un builder	Media	Mitigado por contrato + CSP; frágil sin tipos (ver F-DT)
S2	SPOF: todo en un proceso/VPS	Media	Backups; réplica/cluster en roadmap
S3	Secretos en <code>.env.production</code> del VPS	Baja	Permisos de archivo; rotación recomendada
S4	Sesión de 30 días sin rotación activa	Baja	Revocación por suspensión + fingerprint de contraseña
S5	Sin WAF/fail2ban en el VPS	Baja	Rate-limit en app; Nginx; aceptable en staging

**Fortalezas**

Contraseñas con scrypt, cookies httpOnly firmadas y vinculadas a la contraseña, verificación de firma de webhooks, authz por relación para menores, contenedor sin privilegios y superficie de dependencias mínima.



APÉNDICE G

Hallazgos de Rendimiento

#	Hallazgo	Impacto	Recomendación
P1	<code>/aula</code> ~206 kB First Load (sin code-split)	Medio	Split por ruta; cargar pantallas opcionales bajo demanda
P2	Re-render por <code>innerHTML</code> completo	Bajo	Aceptable; migración incremental a componentes si crece
P3	VPS de 1 vCPU	Medio	Vertical scaling + PgBouncer/Redis antes de 3.000 usuarios
P4	~10–15 s de downtime por deploy con cambio	Bajo	Mitigado con grace period; rolling/health en roadmap



APÉNDICE H

Deuda Técnica

ID	Deuda	Origen	Prioridad
DT-1	Doble schema Prisma mantenido a mano	ADR-0002	Alta
DT-2	UI sin type-safety (<code>@ts-nocheck</code> en <code>scr-*.ts</code>)	ADR-0004	Media
DT-3	Sin code-splitting en el Aula	ADR-0004	Media
DT-4	Estado global mutable sin reactividad	ADR-0004	Baja
DT-5	Migraciones por <code>db push</code> (sin historial versionado)	—	Media

APÉNDICE I

Mejoras Futuras

Horizonte	Iniciativa
Corto	Single source of truth de Prisma (sólo Postgres + Compose local en dev)
Corto	Code-splitting por ruta en el Aula; rebajar First Load
Medio	Migración incremental de pantallas a componentes React tipados
Medio	PgBouncer + Redis + cluster Node para soportar 3.000+ usuarios
Medio	Observabilidad: logs estructurados + métricas + alertas
Largo	Réplica de BD / failover para eliminar el SPOF



APÉNDICE J

Referencias

- Repositorio — <https://github.com/YesIIBr0/OTR> (commit [e1f7bf6cd1daa413b38f26e305b678a1e1be1e30](#))
- [docs/adr/0001..0004](#) — Architecture Decision Records
- [docs/DATA_MODEL.md](#) , [docs/SYSTEM_MAP.md](#) — documentación interna
- [DEPLOY.md](#) — guía de despliegue (Hostinger / Docker Compose)
- Next.js App Router · Prisma ORM · Stripe API · Cloudflare Stream — documentación de proveedores

— Fin del informe —

OTR Academy · Architecture Report v1.0 · e1f7bf6 · Confidencial